

Материали за подготовка — теория и примери

Предмет: Операционни системи (ИУЧ – СПП) · **Клас:** 11 б

Контролна работа 2 — теми: процеси и планиране; контекст и комуникация между процеси; виртуална памет; потребители, групи и права; SSH, FTP/SCP; DNS и DHCP; управление на услуги и планиране на задачи (cron).

Съдържание

1. Процеси: пакетна обработка, многозадачност, времоделене
2. Състояния на процес; създаване и унищожаване
3. Превключване на контекста
4. Сътрудничество и комуникация между процеси (IPC)
5. Виртуална памет
6. Потребители, групи и права към файл
7. SSH и ключове
8. FTP и SCP (и кратко за SFTP)
9. DNS и DHCP като мрежови услуги
10. Управление на услуги: systemd и systemctl
11. Планиране на задачи: cron

1. Процеси: пакетна обработка, многозадачност, времоделене

Процес е програма в изпълнение: има собствен контекст (код, данни, отворени ресурси, идентификатор PID) и се управлява от ОС чрез планировчик.

1.1. Пакетна обработка (batch)

Задачите се подават като **поредица/пакет**; системата ги изпълнява с **минимална или никаква интерактивна намеса** между стъпките на оператора по време на обработката. Подходящо за масови изчисления, отчети, нощни задачи.

Пример (идеен): пакет от десет скрипта за обработка на оценки се пуска веднъж; операторът не отговаря на въпроси между всеки скрипт — работата върви до края на пакета.

1.2. Многозадачност (multitasking)

Няколко програми изглеждат активни „едновременно“, защото ОС **бързо редува** кой процес ползва процесора (кванти време). На едно ядро в даден момент обикновено се изпълнява един поток от инструкции; усещането за паралелизъм идва от скоростта на превключване.

1.3. Времоделене (time-sharing)

Специален случай на многозадачност, ориентиран към **много интерактивни потребители**: процесорното време се дели на **кратки интервали**, за да остане системата отзивчива (терминали, сесии, графичен

работен плот).

Сравнение (обобщение)

Идея	Акцент	Типичен потребител
Пакетна обработка	Поредица задачи без постоянна интеракция	Оператор/система
Многозадачност	Няколко процеса споделят CPU чрез планиране	Един или много
Времеделене	Кратки квантове за интерактивност	Много потребители

2. Състояния на процес; създаване и унищожаване

2.1. Типични състояния (учебен модел)

- **Нов (new)** — създаден е дескрипторът, но още не е готов за изпълнение или чака ресурс.
- **Готов (ready)** — може да се изпълнява, чака ред от планировчика.
- **Изпълняващ се (running)** — инструкциите му се изпълняват на процесор в момента.
- **Чакащ (waiting/blocked)** — чака събитие: вход/изход, семафор, съобщение и т.н.
- **Приключил (terminated)** — приключил е; ресурсите се освобождават от ОС.

2.2. Създаване и унищожаване

Създаване: родителски процес (напр. shell) иска от ядрото да създаде дъщерен процес — заделя се структура (PCB), адресно пространство или споделяне на код, наследяват се част от средата (отворени файлове, променливи на средата).

Унищожаване: при изход програмата предава код на завършване; ядрото освобождава PID, памет, дескриптори на файлове; родителят може да прочете статуса (в Unix чрез `wait`).

Пример (Linux): от обвивката команда като `ls` стартира **нов процес**; след извеждане на списъка той завършва и shell-ът продължава.

3. Превключване на контекста

Превключване на контекста е запазване на „снимка“ на текущия процес (регистри, указател към инструкция, информация за стека, таблици за страници частично и т.н.) и зареждане на запазена снимка на друг процес, за да продължи той. Така планировчикът може да редува процесите често.

Връзка с многозадачност: без превключване на контекста е невъзможно да се редуват процесите прозрачно за потребителя в едно ядро.

4. Сътрудничество и комуникация между процеси (IPC)

IPC (inter-process communication) позволява на процесите да обменят данни и да се синхронизират.

Механизъм	Кратко	Примерна употреба
Тръба (pipe)	Еднопосочен поток байтове между свързани процеси	<code>ls grep txt</code>
Именувана тръба (FIFO)	Тръба с име във файловата система	клиент/сървър на една машина
Съобщения (message queue)	Опашка от съобщения с тип/приоритет	координация на услуги
Споделена памет	Общ сегмент — много бърз обмен	ниска латентност; изисква синхронизация
Сокет	Крайна точка за мрежа или локално (Unix domain)	клиент–сървър по TCP/UDP
Сигнал	Кратко уведомление (<code>SIGTERM</code> , <code>SIGINT</code> ...)	спиране, прекъсване, презареждане

Пример с тръба:

```
grep "ERROR" /var/log/syslog | wc -l
```

Първият процес филтрира редове; вторият брои редовете — данните минават през тръба без междинен файл.

Риск: при споделен ресурс (файл, памет) без заключване могат да възникнат **състезателни събития** — два процеса четат/пишат едновременно и резултатът става непредсказуем.

5. Виртуална памет

Всяка програма вижда **логическо (виртуално) адресно пространство**, по-голямо от наличната RAM. ОС картографира виртуални адреси към **рамкове** в физическа RAM или към блокове на **вторично устройство** (swap/страницен файл), използвайки **страници** и **таблици на страници**.

- **Страница навътре / навън:** при нужда се зареждат нужните страници в RAM („lazy“ подход).
- **Page fault:** достъп до страница, която още не е в RAM — ядрото я зарежда или обработва грешка.
- Ползи: изолация между процеси, повече логическа памет от физическа, общи библиотеки между процеси.

Аналогия: много книги на рафта ви (диск); на бюрото имате само няколко разтворени (RAM). Превключвате страниците, които реално четете — останалото остава „на рафта“, докато не потрябва.

6. Потребители, групи и права към файл

В Unix-подобните системи всеки файл има **собственик (user)** и **група (group)**. Правата се описват с **триада** за собственик, група и останали (others): **read, write, execute**.

Пример: изход от `ls -l`

```
-rwxr-x--- 1 ana students 4096 map 05 10:01 skript.sh
```

Собственикът `ana` може да чете, пише и изпълнява; членовете на група `students` — само да четат и изпълняват; останалите — без права.

Символно: `chmod u+x file` добавя право за изпълнение за собственик.

Числено: `chmod 750 skript.sh` означава `rwx` (7) за собственик, `r-x` (5) за група, `---` (0) за други.

```
sudo useradd -m nov_potrebitel
sudo groupadd razrabotchici
sudo usermod -aG razrabotchici nov_potrebitel
```

Командите за потребители/групи изискват администраторски права; на учебна машина използвайте с повишено внимание.

7. SSH и ключове

SSH (Secure Shell) осигурява криптиран отдалечен достъп (обикновено до shell). Вместо парола в открит вид често се използва **асиметрична двойка ключове**: частен ключ остава при клиента (защитен с права!), публичният ключ се записва на сървъра (напр. в `~/.ssh/authorized_keys`).

```
ssh-keygen -t ed25519 -C "uchenik@uchilishte"
ssh-copy-id uchenik@192.168.1.50
ssh uchenik@192.168.1.50
```

Частният ключ никога не се качва в чат или публично хранилище. При компрометиране — генерирайте нова двойка и заменете публичния ключ на сървъра.

8. FTP и SCP (и SFTP)

Услуга	Основна идея	Бележки
FTP	Класически файлов трансфер клиент-сървър	Често нешифриран канал (FTP); има FTPS/FTP над TLS
SCP	Копиране на файлове през SSH	Използва SSH за шифриране и удостоверяване
SFTP	Файлови операции като подсистема на SSH	Интерактивни сесии, резюме, списъци

```
scp /tmp/izpit.txt uchenik@host:/home/uchenik/dokumenti/  
scp uchenik@host:/etc/hostname ./hostname.remote
```

9. DNS и DHCP

- **DNS** — система имена към номера (хост име → IPv4/IPv6). Примери записи: A, AAAA, MX, CNAME.
- **DHCP** — автоматично раздаване на мрежова конфигурация: IP адрес, маска, шлюз по подразбиране, DNS сървъри, време на наем.

Пример: на лаптоп в клас Wi-Fi достъпът често идва чрез DHCP — получавате адрес без ръчно въвеждане на IP. Когато пишете `pvq.bg` в браузър, приложението пита DNS кой числов адрес да използва.

```
resolvectl query example.com      # systemd-resolved при някои дистрибуции  
ip addr show                     # локални адреси  
cat /etc/resolv.conf              # наблюдавайте DNS конфигурация (може да е  
управлявана автоматично)
```

10. Управление на услуги: systemd и systemctl

В модерните Linux дистрибуции услуги (демони) често се управляват с **systemd**. `systemctl` стартира, спира, проверява състояние и настройва автоматично стартиране.

```
sudo systemctl status ssh  
sudo systemctl start apache2  
sudo systemctl stop apache2  
sudo systemctl restart nginx  
sudo systemctl enable cups      # включва при зареждане  
sudo systemctl disable cups     # без автоматичен старт при boot
```

Имената на услугите са различни по дистрибуции (`ssh` vs `sshd` , `apache2` vs `httpd`).

11. Планиране на задачи: cron

cron изпълнява команди по разписание. Потребителският crontab използва пет полета за време (минута, час, ден от месеца, месец, ден от седмицата), следвани от командата.

Поле	Позволенни стойности
Минута	0–59
Час	0–23
Ден от месеца	1–31
Месец	1–12
Ден от седмицата	0–7 (0 и 7 = неделя при много конфигурации)

```
crontab -e # редактирайте личното разписание
# всеки ден в 06:30
30 6 * * * /home/uchenik/backup.sh
# на всеки 15 минути
*/15 * * * * date >> /home/uchenik/cron-heartbeat.log
```

Пример: системен часовник за синхронизация може да ползва задача на root; за учебни цели често е достатъчно да разберете къде се задава командата и как се изразява интервалът.

